Least-norm Application: Signal Reconstruction

Imagine that you are monitoring a sensor signal (like a voltage or a current wave). Due to hardware glitches, you only captured a few specific data points at different times. You need to reconstruct the original signal $x(t)$ assuming that it is a combination of harmonic frequencies, but there are infinitely many signals that could pass through these three points. The **best** reconstruction is the one that satisfies the measurements with the least signal energy (minimizing the 2-norm of the coefficients).

Suppose that the signal is modeled as a sum of sines:

$$x(t) = \sum_{k=1}^{n} c_k \sin(k\omega_0 t)$$

Let $S = \{t_1, t_2, \ldots, t_m\}$ be the set containing time indices of sample points.

1. The constraint is that the signal must pass through m observations where $m < n$: $Ac = y$, where y is the vector of observed values $x(t_i)$ for $i = 1, 2, \ldots, m$ and A is the matrix of sine basis functions evaluated at the sample times. Explain how to determine $A \in \mathbf{R}^{m \times n}$ and $y \in \mathbf{R}^m$.
2. The objective is to estimate c based on the principle that we minimize $\|c\|_2$ subject to $Ac = y$.
3. For your own study, plot the reconstructed signal versus the ground-truth (`y_true_fine`) and discuss the results. Submit this in the paper homework.

Problem

Write a python function to take the signals at coarse resolution specified by $t_{\text{fine}}, y_{\text{samples}}$ to construct the least-norm problem $Ac = y$, and compute the solution c. Use the estimate to reconstructed the signal at finer resolution, specified by t_{fine}. Use the number of sine coefficients $n_{\text{coeffs}} = 10$. To do so, use dataset specified in the file `leastnorm_signal_recon.npz`. Then, submit the function to Grader.

Implementation

Goal: Complete the code as the problem state and submit to the Grader.

```
import numpy as np

def leastnorm_signal_recon(t_samples, y_samples, n_coeffs, t_fine):
    ...
    return c, A, y, y_reconstructed

def load_npz_data(filename):
    ...
    return t_samples, y_samples, t_fine, y_true_fine

if __name__ == "__main__":
    import matplotlib.pyplot as plt
    # Load the dataset
    t_samples, y_samples, t_fine, y_true_fine = ...
    n_coeffs = 10
    c, A, y, y_reconstructed = leastnorm_signal_recon(t_samples, y_samples, n_coeffs,
    ↪  t_fine)
    # Visualization
    # Then try plot your reconstructed signal against the true signal.
```

Constrains

Inputs:

- t_{samples} is an array of m time points.
- y_{samples} is an array of m-length signals y that the reconstruction has to match.
- n_{coeffs} is the number of sine components to be included in the model.
- t_{fine} is an array of T time points (finer resolution) that is used to reconstruct the signal.

Outputs:

- c is an array of length n_{coeffs}.
- A, y are the parameters of system $Ac = y$ in the least-norm problem
- $y_{\text{reconstructed}}$ is the reconstructed signal $y(t)$ by using t_{fine}